

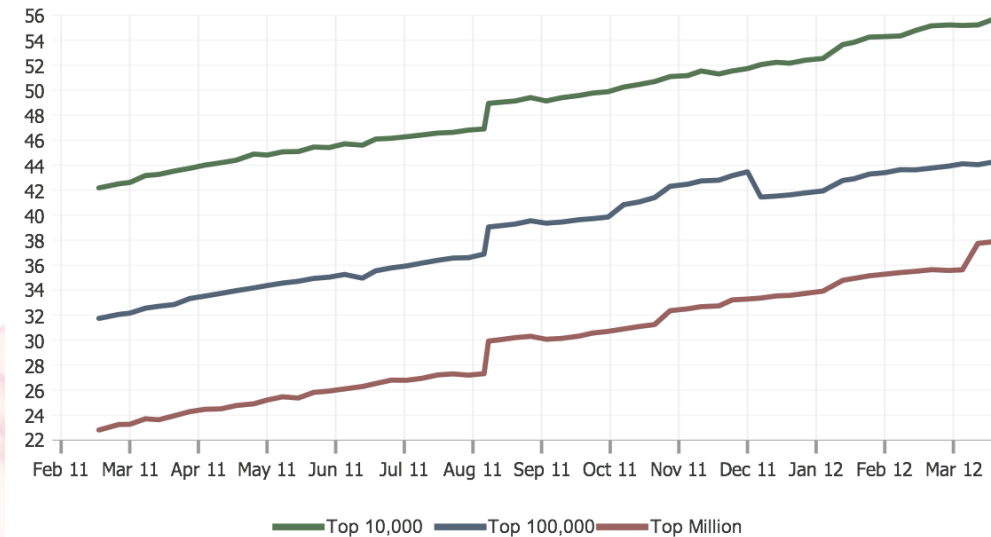


Российские
интернет-технологии
2012

Профилирование и оптимизация jQuery-кода

Владимир Журавлев

Самый популярный фреймворк*



* По данным builtwith.com



Российские
интернет-технологии
2012

Все любят jQuery

- Простота и более низкий порог вхождения;
- кроссбраузерный фасад для рутинных операций;
- наличие плагинов практически на все случаи жизни.



**ЖУТКО ТОРМОЗИТ.
ХОЧЕТСЯ ВЗЯТЬ И ПЕРЕПИСАТЬ!**



Российские
интернет-технологии
2012



javascript optimization



Поиск

Результатов: примерно 40 300 000 (0,16 сек.)

Все результаты

Картинки

Карты

Видео

Новости

Ещё

Санкт-Петербург

Изменить место

Весь Интернет

Только на

русском

Перевод

результатов

За всё время

За час

JavaScript Optimization

home.earthlink.net/~kendrasg/info/js_opt/ - Перевести эту

One, it is intended to be a compendium of **JavaScript optimization** techniques, a place where one can turn to find optimization examples, some well known to ...

Оптимизация Javascript-кода

javascript.ru > Оптимизация

Основные узкие места - как правило, там, где **javascript** вызывается очень часто. Мы рассмотрим основные причины тормозов и то, как их преодолеть.

JavaScript. Оптимизация: опыт, проверенный временем ...

habrahabr.ru/post/137318/

31 янв 2012 – Предисловие Давно хотел написать. Мысли есть, желание есть, времени нету... Но вот нашлось, так что привет, Хабра. Здесь я собрал ...

Оптимизация javascript скриптов

htmlweb.ru/java/optimizaciya.php

Описание способов ускорения загрузки и выполнения работы **javascript** скриптов.

JavaScript optimization Part 1: Adding DOM elements to document ...

kpumuk.info/javascript/javascript-optimization-part-1-adding-dom-el...

25 мар 2007 – Most web-developers writing tons of **JavaScript**, especially in our Web 2.0 century. It's powerful technology, but most browsers has very slow ...

Table of Contents

Introduction

Speed

IE vs. Netscape

About the Tests

Straight Forward

- Limit Work Inside Loops
- Minimize Repeated Expressions
- Unnecessary Variables
- If-Then-Else Structure

Not So Straight Forward

- Switch Vs. If-Then-Else
- Switch Structure
- Bit-Shifting
- Look Up Tables
- Loop Unrolling
- Reverse Loop Counting
- Loop Flipping
- JS Math

Straight as a Pretzel

- Duff's Device
- Faster Duff's Device

Size

Coming Soon

Memory

Coming Soon

JavaScript Optimization

Introduction

This document is intended to serve a number of purposes. One, it is intended to be a compendium of JavaScript optimization techniques, a place where one can turn to find optimization examples, some well known to JavaScript Programmers, and some not so well known. Two, it is also a source of performance information with regard to how these techniques perform in comparison to each other. Lastly, it is intended to address the issue of which programming optimization techniques work as intended in the interpreted environment in which JavaScript code runs.

There are a few web sites out there that deal with JavaScript optimization, but most only deal with size issues, and if they do deal with other issues, it is in a very general way. I seek to rectify that here. I will also delve into some of the more obscure issues related to programming optimization, and how they might impact on JavaScript. This document is divided into three main sections: *Speed*, *Size* and *Memory*.

I have chosen JavaScript 1.3 as the basis for the tests.

- *Jeff Greenberg*

Speed

I am starting with this section because it is one of the most neglected aspects of JavaScript optimization. Of course, most of the time it is not needed in everyday JavaScript. There are some things to keep in mind about speed optimization:

- 1) The best kind of speed optimization is more efficient algorithm design. Before you go nuts trying to squeeze every last ounce of speed out of every statement, take a serious look at the design of your algorithms first. Why? See next point.
- 2) Speed optimization is usually the last step and the last resort while programming. Code that has been altered explicitly for speed concerns is almost always difficult to read, maintain and overhaul. If you plan on sharing your code with others, this is something you should think about seriously.
- 3) For most uses of JavaScript, speed optimization is just not necessary. If you are feeling the urge to speed up your code just for the sake of it, don't. It's not worth it. If, however, speed is critical to your project, and you can't seem to get the necessary speed from your design, then your

- loop flipping;
- array-push-join вместо сложения строк;
- loop unrolling;
- {'Вася': 1, 'Коля': 1}[name]
- ~~ (0.5 + n)



Преждевременная оптимизация

Преждевременная оптимизация

Симптомы:

- оптимизация кода еще до того как он начнет тормозить.

Последствия:

- потеря времени;
- ухудшение читаемости.



Экономия на спичках

Экономия на свичках

Симптомы:

- оптимизация частей кода, принципиально не влияющего на производительность;
- автоматическое использование «оптимальных» конструкций.

Последствия:

- потеря времени;
- потеря читабельности;
- легко упустить настоящую оптимизацию.





Профилирование

Как измерять?

- Таймеры (например, `console.time`);
- профилирование JS (Firebug, Web Inspector, YUI Profiler);
- анализ timeline (Web Inspector).

Таймеры

Зачем:

- убедиться, что данный блок кода тормозит;
- постоянно мониторить производительность модулей;
- быстро оценить успешность оптимизации.

Чем замерять:

- `console.time`;
- любой самописный таймер.

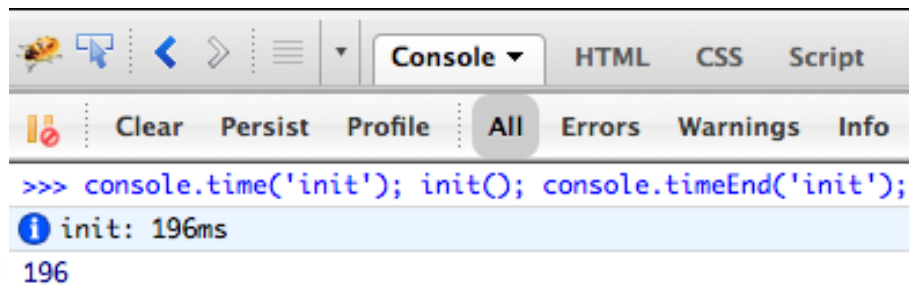
console.time

Использование

```
console.time('init');
```

```
/* блок измеряемого кода */
```

```
console.timeEnd('init');
```



Профилирование JS

Зачем:

- для получения полного отчета о том на что тратит время ваш js код.

Чем профилировать:

- console.profile;
- YUI profiler;
- любой другой самописный профайлер.



console.profile

Достоинства:

- есть в Firefox, Chrome, Safari, IE 8+;
- достаточный минимум.

Недостатки:

- много шума при профилировании jQuery-кода;
- разные форматы отчетов.

console.profile

Использование

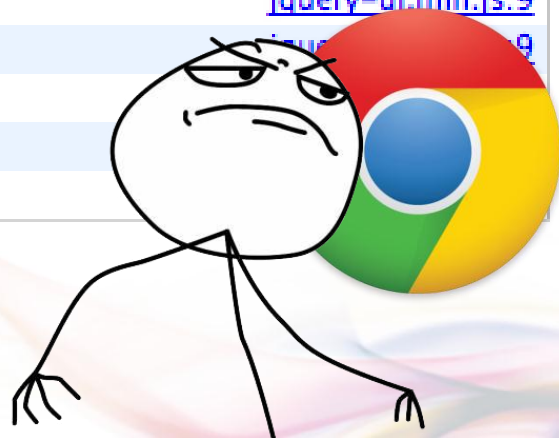
```
console.profile('init');
```

```
/* блок измеряемого кода */
```

```
console.profileEnd('init');
```



Self ▼	Total	Function
10.35s	10.35s	(program)
48ms	48ms	(garbage collector)
34ms	43ms	▼ http://code.jquery.com/ui/1.8.18/jquery-ui.min.js
1ms	2ms	▼ (anonymous function) jquery-ui.min.js:12
0	1ms	▼ Datepicker jquery-ui.min.js:12
0	1ms	▼ e jquery.min.js:2
0	1ms	▼ e.fn.e.init jquery.min.js:2
1ms	1ms	e.extend.merge jquery.min.js:2
0	1ms	▼ (anonymous function) jquery-ui.min.js:9
0	1ms	▼ e.extend.each jquery.min.js:2
1ms	1ms	a.ui.version.a.extend.data jquery-ui.min.js:9
0	1ms	▼ (anonymous function) jquery-ui.min.js:9
0	1ms	► f.each.f.fn.(anonymous function)
0	1ms	▼ (anonymous function)
0	1ms	▼ a.widget

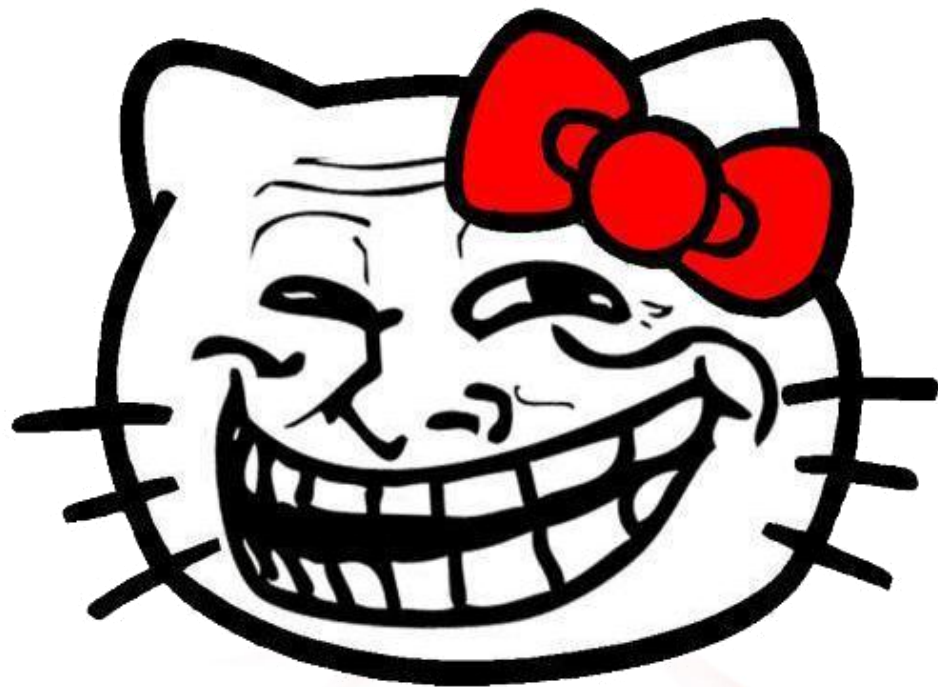


Function	Calls	Percent ▼	Own Time	Time	Avg	Min	Max	File
anonymous	112	11.66%	29.798ms	49.563ms	0.443ms	0.017ms	21.228ms	jquery.min.js (line 5)
anonymous	40	7.84%	20.035ms	72.211ms	1.805ms	0.012ms	65.031ms	jquery.min.js (line 5)
anonymous	332	6.15%	15.712ms	26.697ms	0.08ms	0.036ms	0.934ms	jquery.min.js (line 4)
anonymous	524	5.81%	14.838ms	17.374ms	0.033ms	0.002ms	1.123ms	jquery.min.js (line 3)
anonymous	491	4.02%	10.273ms	12.087ms	0.025ms	0.013ms	0.464ms	jquery.min.js (line 5)
anonymous	1	3.81%	9.731ms	14.065ms	14.065ms	14.065ms	14.065ms	



Function	Count	Inclusive Time (ms)	Exclusive Ti...	URL
JScript - window script block	183	320,46	320,46	http://jqueryui.com/js/demos.js
clean	40	320,46	300,43	http://ajax.googleapis.com/ajax/libs/jquery/1.7....
data	635	280,40	200,29	http://ajax.googleapis.com/ajax/libs/jquery/1.7....
hidden	8	140,20	140,20	http://ajax.googleapis.com/ajax/libs/jquery/1.7....
callback	183	450,65	130,19	
type	1 555	110,16	110,16	http://ajax.googleapis.com/ajax/libs/jquery/1.7....
style	490	130,19	100,14	http://ajax.googleapis.com/ajax/libs/jquery/1.7....
extend	525	270,39	90,13	http://ajax.googleapis.com/ajax/libs/jquery/1.7....
html	26	100,14	90,13	http://ajax.googleapis.com/ajax/libs/jquery/1.7....
trigger	10	260,37	90,13	http://ajax.googleapis.com/ajax/libs/jquery/1.7....
e	395	640,92	80,12	http://ajax.googleapis.com/ajax/libs/jquery/1.7....
isArray	544	90,13	80,12	http://ajax.googleapis.com/ajax/libs/jquery/1.7....
init	395	560,81	70,10	http://ajax.googleapis.com/ajax/libs/jquery/1.7....
acceptData	656	70,10	70,10	http://ajax.googleapis.com/ajax/libs/jquery/1.7....
isFunction	419	90,13	60,09	http://ajax.googleapis.com/ajax/libs/jquery/1.7....
isPlainObject	485	100,14	50,07	http://ajax.googleapis.com/ajax/libs/jquery/1.7....
_data	620	320,46	50,07	http://ajax.googleapis.com/ajax/libs/jquery/1.7....
add	332	290,42	40,06	http://ajax.googleapis.com/ajax/libs/jquery/1.7....
camelCase	723	50,07	40,06	http://ajax.goo
bB	116	40,06	40,06	http://ajax.g
JScript - window script block	492	170,25	40,06	http://ajax.
nodeName	46	40,06	40,06	http://ajax.





Российские
интернет-технологии
2012

YUI Profiler

Зачем:

- для браузеров, в которых нет профайлера;
- чтобы профилировать только обращения к jQuery;
- для мониторинга производительности модулей.

developer.yahoo.com/yui/profiler



Российские
интернет-технологии
2012

YUI Profiler

Использование:

```
YAHOO.tool.Profiler.registerFunction('init', window);
```

```
// профилируемый код
```

```
init();
```

```
var report = YAHOO.tool.Profiler.getFunctionReport('init');
```

```
YAHOO.tool.Profiler.unregisterFunction('init');
```



Российские
интернет-технологии
2012

YUI Profiler

Достоинства:

- работает везде;
- хорошо подходит для профилирования jQuery (например, можно логировать только `$.fn.*`, `$.*`).

Недостатки:

- нельзя отличить `$(selector)` от `$(html)`, `$(element)`, `$(function)`;
- логирует вложенные вызовы, что может исказить статистику;
- нет own time, только total time.

jquery.profile.js

Зачем:

- потому что велосипеды — это здорово;
- чтобы избавиться от недостатков YUI Profiler.

Плюсы:

- не логирует вложенные вызовы;
- отдельно логирует \$(selector), \$(html), \$(function) и \$(element).



github.com/private-face/jquery.profile



Российские
интернет-технологии
2012

jquery.profile.js

Использование:

```
var profiler = new $.Profiler;
```

```
profiler.start();
```

```
init();
```

```
profiler.stop();
```

```
profiler.topTime();
```



Российские
интернет-технологии
2012

▼ [Profiler] Total profiled jQuery time: 61ms				
Name	Time	Calls	Top callers	
\$(selector)	10ms	101	2ms	.bb-wizard
			1ms	audio
			1ms	.datetime-control
			2ms	#account-name
			1ms	.banner-checkbox input
\$.fn.delegate	8ms	25	2ms	function()
			1ms	function()
			1ms	function()
\$(ready)	7ms	32	2ms	function()
			1ms	function()
			1ms	function()
\$(html)	6ms	14	3ms	
			1ms	<div id="fancybox-tmp"></div>
			1ms	<div id="fancybox-outer"></div>
			1ms	<a href="javascript:;" id="fancybox-ri
\$.fn.append	6ms	3	6ms	function()
\$.extend	5ms	20	2ms	function()



jquery.profile.js

Недостатки:

- не логирует вложенные вызовы;
- «beta than nothing».



Анализ timeline

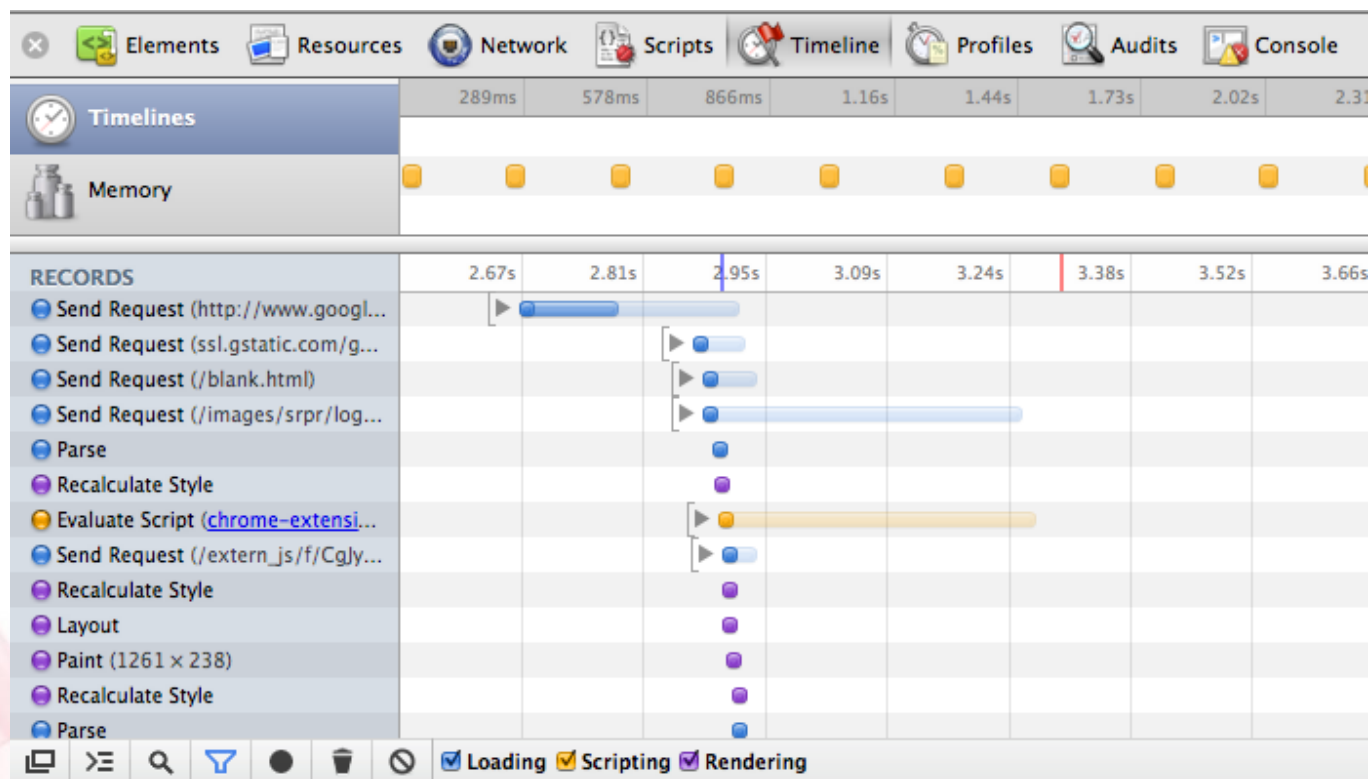
Зачем:

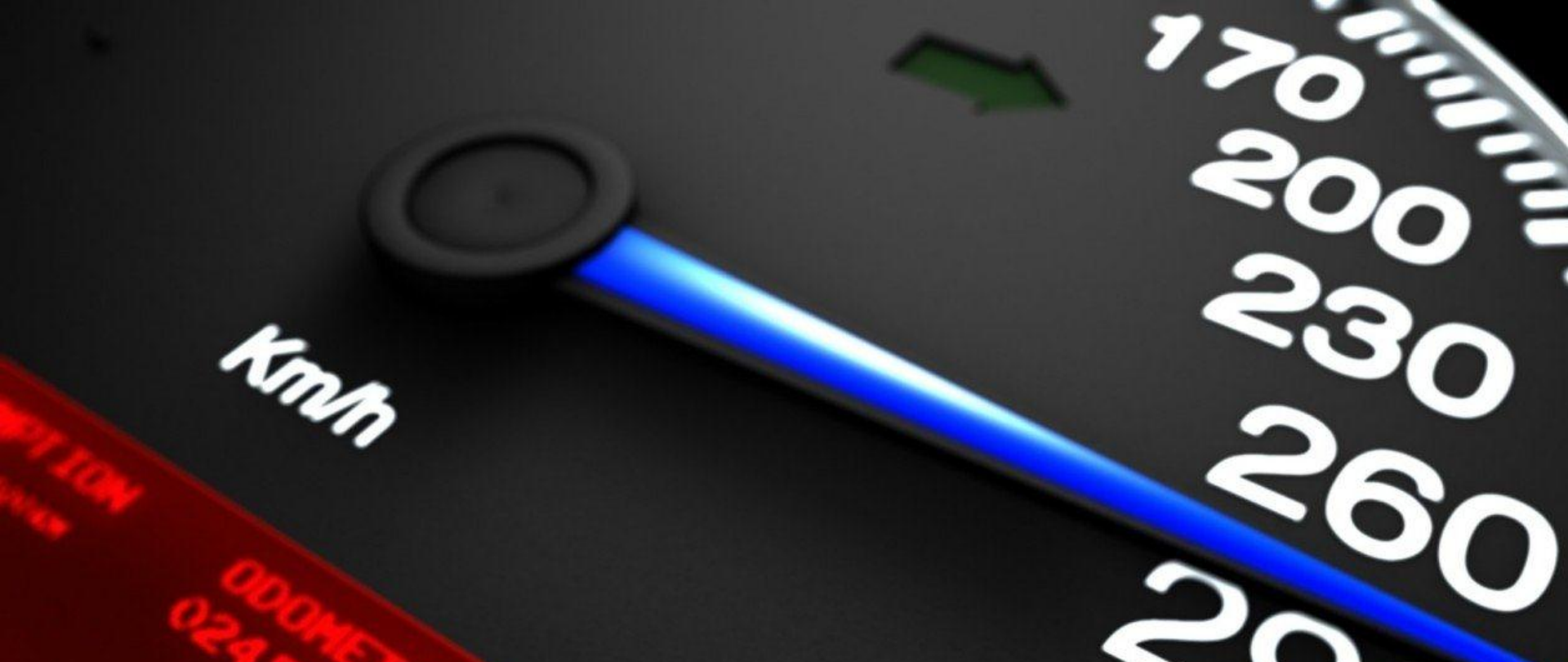
- чтобы определить как рендеринг влияет на производительность.

Что использовать:

- Chrome Web Inspector.







Оптимизация

```
$('#div').click(function() {  
    var div = $(this);
```

```
    // ...
```

```
});
```

```
$('#div').each(function() {  
    var div = $(this);
```

```
    // ...
```

```
});
```



Селекторы

Симптомы:

- Большое число обращений к `$(selector)`, `$.fn.find`, и т.п.;
- Некоторые выборки выполняются довольно долго.

1. Селекторы разбираются справа-налево

`$('#div.post a.vote-up')` // плохо

`$('.post .vote-up');` // лучше

`$('.vote-up');` // еще лучше



2. Сужайте область поиска

- Задавайте контекст

<code>\$('div.post a.vote-up');</code>	<code>// плохо</code>
<code>\$('#content').find('.vote-up');</code>	<code>// хорошо</code>

- Используйте children, closest и т.п.

<code>\$('ul#list').find('li');</code>	<code>// плохо</code>
<code>\$('ul#list').children('li');</code>	<code>// хорошо</code>

<code>\$('li').parents('div').first();</code>	<code>// плохо</code>
<code>\$('li').closest('div');</code>	<code>// хорошо</code>



3. Кешируйте выборки и используйте chaining

```
$('div.post').mousemove(function() ...);
```

```
$('div.post').find('a').click(function() ...);
```

// плохо

```
var posts = $('div.post');
```

```
posts.mousemove(function() ...);
```

```
posts.find('a').click(function() ...);
```

// хорошо

```
$('#div.post')
```

```
  .mousemove(function() ...)
```

```
  .find('a').click(function() ...)
```

```
  .end()
```

```
  .find('.invisible').hide();
```

// тоже хорошо



4. Делегируйте обработку событий

- Избавьтесь от live. Совсем.

```
$('#a').live('click', function() {...
```

// плохо

```
$(document).on('click', 'a', function() {...
```

// супер

- Замените bind на on/delegate

```
$('.vote').click(function() {...
```

// плохо

```
$('#content').on('click', '.vote', function() {...
```

// хорошо



Слишком много on/delegate

Симптомы:

- задержка между событием и реакцией на него;
- большое количество вызовов \$.fn.is в профайлере.

Оптимизация:

- отсекать лишние on/delegate;
- использование bind вместо on/delegate.



Повторяющиеся события

Симптомы:

- «тормоза» в момент ресайза, скролла или интенсивного ввода текста;
- профайлер показывает большое количество вызовов обработчиков событий `keydown`, `scroll`, `resize` и т.п.

Оптимизация:

- Группировка нескольких вызовов события в один путем применения декораторов `debounce` и `throttle`.

benalman.com/projects/jquery-throttle-debounce-plugin/



debounce

`$.debounce`

- Группирует повторяющиеся события интервал между которыми меньше определенного значения.

события:



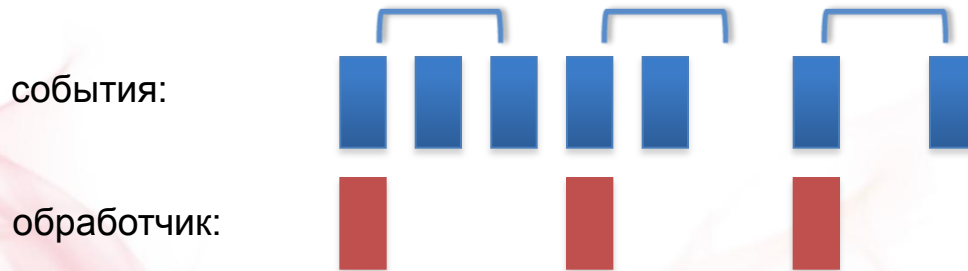
обработчик:



throttle

\$.throttle

- Группирует события которые были вызваны в пределах одного интервала.



Оптимизация \$(element)

Симптом:

- профайлер показывает существенную трату времени на \$(element).

Оптимизация:

- использовать один общий jQuery-объект для всех элементов внутри each/map;
- [quickEach](#).



```
// создаем общий jQuery-объект за пределами map/each:
```

```
var a = $([0]);
```

```
$('#a').each(function() {
```

```
    // подставляем текущий элемент в объект
```

```
    a[0] = this;
```

```
    // и используем его как обычный $(this)
```

```
    // ...
```

```
});
```



Недостаток:

- Нельзя использовать во вложенных асинхронных функциях.

```
var a = $([0]);
```

```
$('#a').each(function() {  
    a[0] = this;
```

```
    a.click(function() {
```

```
        // всегда будет выводиться текст последней ссылки
```

```
        alert(a.text());
```

```
    });
```

```
});
```



Оптимизация создания элементов

Симптом:

- профайлер показывает, что много времени уходит на `$(html)`, `append`, `prepend`, `before`, `after` и т.п.

1. Клонирование быстрее создания

// создание

```
var soldiers = $.map(names, function(name) {  
    return $('<div class="soldier"><span>' + name + '</span></div>');  
});
```

// клонирование

```
var bobaFett = $('<div class="soldier"><span>Boba</span></div>');  
var name = bobaFett.find('span');
```

```
var soldiers = $.each(names, function(name) {  
    name.text(name);  
    return bobaFett.clone();  
});
```



2. Минимизируйте число операций вставки в DOM

// вставка сразу после создания

```
$.each(messages, function(_, text) {  
    $('#history').append(<div class="message">' + text + '</div>');  
});
```

// вставка пакетом

```
var fragment = $(document.createDocumentFragment());
```

```
$.each(messages, function(_, text) {  
    fragment.append(<div class="message">' + text + '</div>');  
});  
$('#history').append(fragment);
```



Restyle, layout, paint

Restyle:

- пересчет стилей не меняющих геометрию объектов.

Layout:

- пересчет геометрии.

Paint:

- перерисовка после пересчета свойств и геометрии.

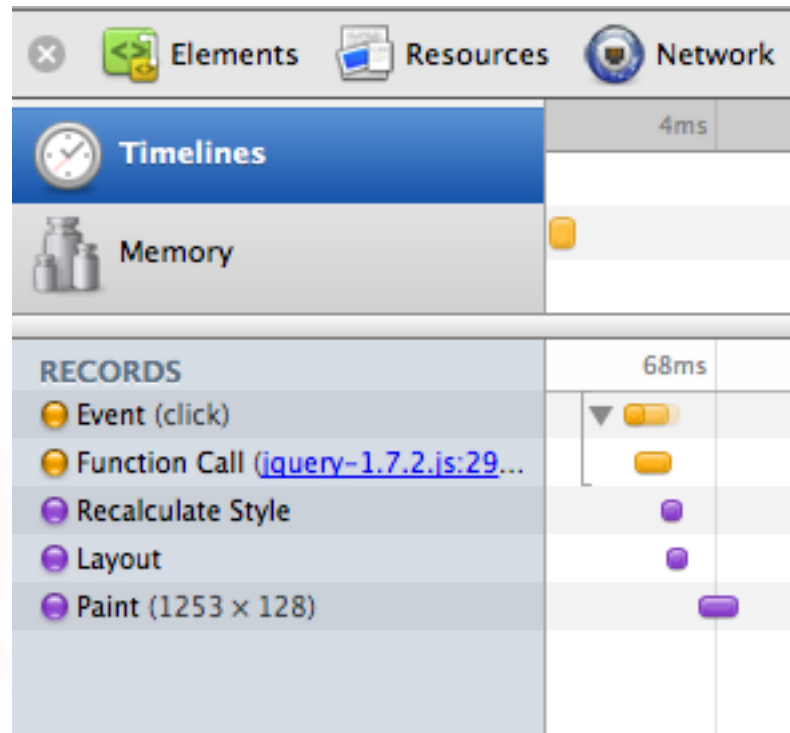


Restyle, layout, paint

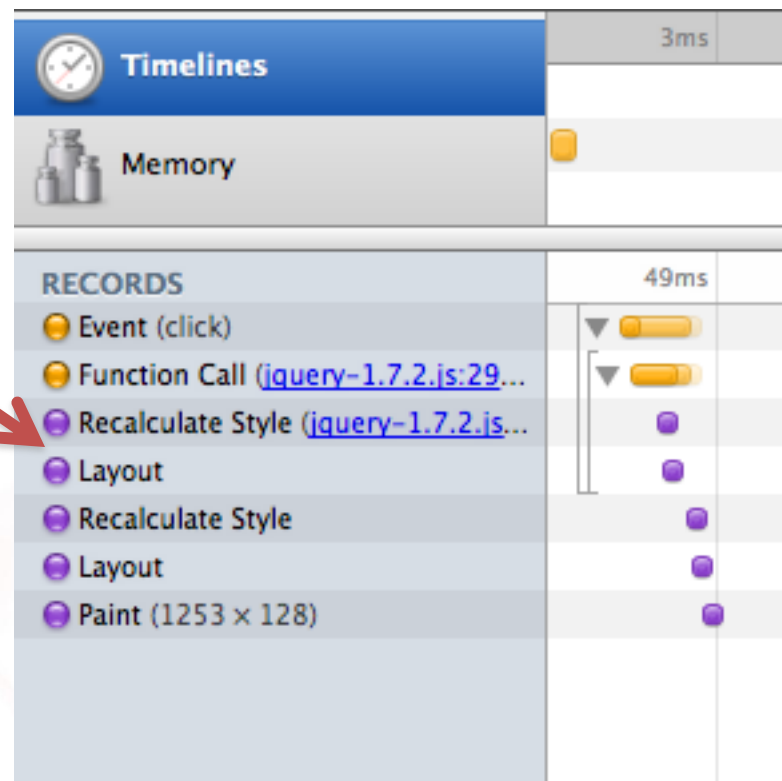
- Изменения откладываются на потом;
 - иногда браузер вынужден применить изменения досрочно:
 - offsetTop/Left/Width/Height
 - scrollTop/Left/Width/Height
 - clientTop/Left/Width/Height
 - getComputedStyle(), currentStyle [IE]
- } **jQuery.fn.css**
- время restyle, layout, paint сильно зависит от разметки.

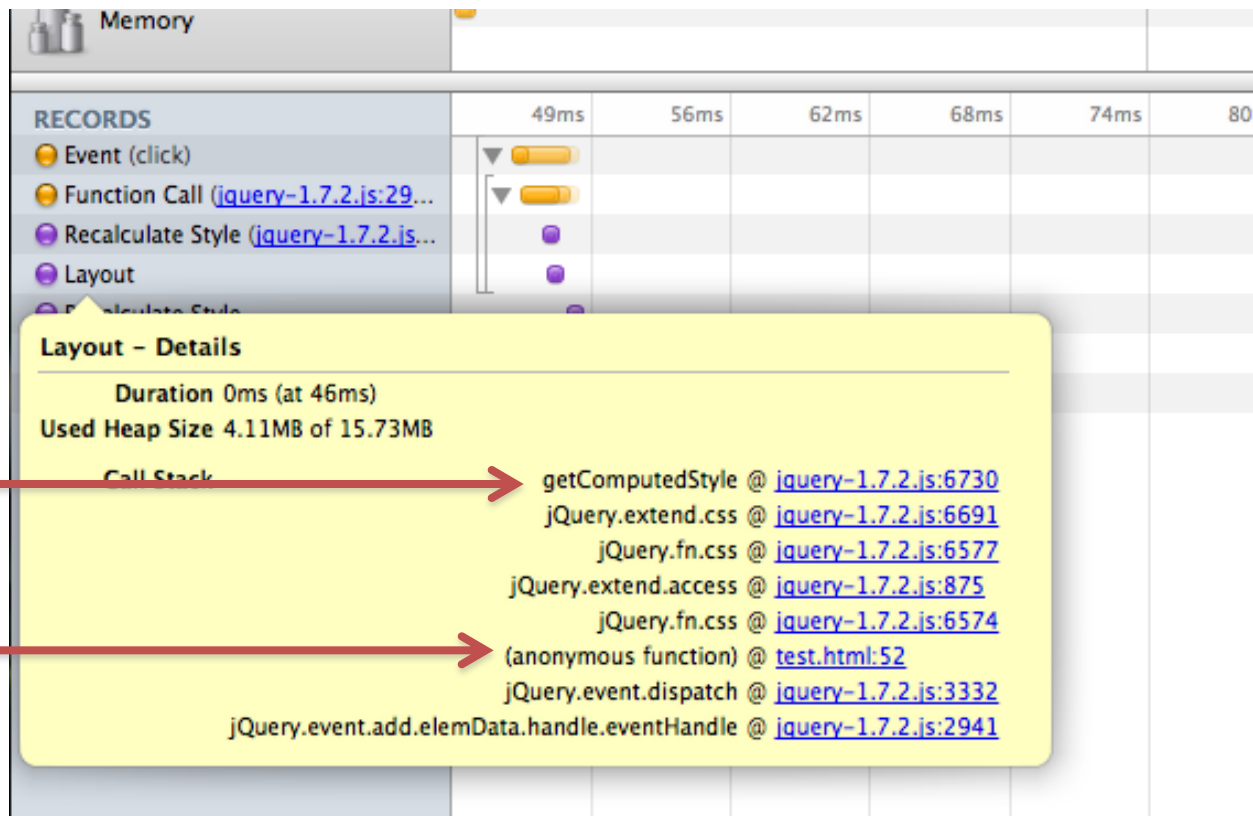


```
$('#div').click(function() {  
    var e = $(this);  
  
    e.css('color', e.css('backgroundColor'));  
  
    e.css('width', 100);  
  
    e.css('font-size', 20);  
  
    return false;  
});
```



```
$('#div').click(function() {  
    var e = $(this);  
  
    e.css('width', 100);  
  
    e.css('color', e.css('backgroundColor'));  
  
    e.css('font-size', 20);  
  
    return false;  
});
```





Оптимизация .css

- Проводить операции с элементами с `display: none`, либо до вставки в документ;
- заменить несколько `css` на `add/removeClass`;
- группировать запросы CSS-свойств в начале функции (когда очередь `restyle` и `layout` пуста);
- не чередовать запросы и установку CSS у элемента.

Инициализация по требованию

Показания к применению:

- При загрузке страницы одновременно инициализируются множество одинаковых контролов (модулей, плагинов).

Оптимизация:

- Инициализировать каждый контрол в момент первого обращения к нему.

Повышение отзывчивости

Симптом:

- В момент выполнения ресурсоемкого кода браузер плохо реагирует на внешние воздействия.

Оптимизация:

- Откладывать трудоемкие итерации через `setTimeout`.

Общие рекомендации

- Кешируйте все, что может пригодиться;
- отсекайте лишние выборки, инициализацию плагинов и модулей, на страницах, на которых они точно не нужны;
- медленный JavaScript — далеко не единственная и не всегда главная причина «тормозов» на сайте;
- не бойтесь пересматривать требования.

При возникновении
любых непонятных
ситуаций читайте
исходники jQuery



Владимир Журавлев

Evil Martians

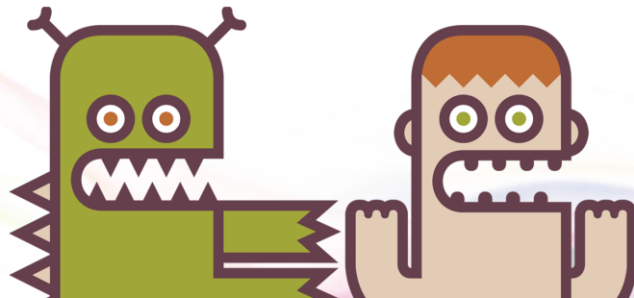
private.face@gmail.com

github.com/private-face

[@private_face](#)



Российские
интернет-технологии
2012





Трёхмерная графика

Материал из Википедии — свободной энциклопедии

[\[править\]](#)

Пожалуйста посетите Доклад Андрея Ситника «Вращай, двигай, загибай» о практике 3D в Вебе



Текущая версия страницы пока не проверялась опытными участниками и может значительно отличаться от версии, проверенной 30 декабря 2011; проверки требуют 5 правок.

Трёхмерная графика (3D Graphics, Три измерения изображения, 3 Dimensions, *рус. 3 измерения*) — раздел компьютерной графики, совокупность приемов и инструментов (как программных, так и аппаратных), предназначенных для изображения объёмных объектов. Больше всего применяется для создания изображений на плоскости экрана или листа печатной продукции в архитектурной визуализации, кинематографе, телевидении, компьютерных играх, печатной продукции, а также в науке и промышленности.

Трёхмерное изображение на плоскости отличается от двумерного тем, что включает построение геометрической проекции трёхмерной модели сцены на плоскость (например, экран компьютера) с помощью специализированных программ. При этом модель может как соответствовать объектам из реального мира (автомобили, здания, ураган, астероид), так и быть полностью абстрактной (проекция четырёхмерного фрактала).

Для получения трёхмерного изображения на плоскости требуются следующие шаги:

- моделирование* — создание трёхмерной математической модели сцены и объектов в ней.
- рендеринг* (визуализация) — построение проекции в соответствии с выбранной физической моделью.
- вывод полученного изображения на устройство вывода — дисплей или принтер.

Однако, в связи с попытками создания 3D-дисплеев и 3D-принтеров, трёхмерная графика не обязательно включает в себя проецирование на плоскость.



Пример 3D-графики